

TUGAS SINKRONISASI

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Nama: Krisna Aji Pratama

NIM: 2410651048

Kelas: B

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025/2026

Daftar Isi

BAB III	3
SOAL	3
3.1 Analisislah Masalah	3
3.2 Implementasi	4
3.2.1 Dining deadlock.....	4
3.2.2 Resource ordering	10
3.2.3 limiting philosophers.....	17
3.3 Testing dan Output.....	24
3.3.1 Dining_deadlock.....	24
3.3.2 Resource ordering	24
3.3.3 Solution limiting	25
3.4 Analisis Hasil.....	27
3.4.1 Dining deadlock.....	27
3.4.2 Resource ordering	27
3.4.3 Solution limiting	27
3.5 Tabel perbandingan efesiensi dan fairness	28
3.6 Analisis mendalam	29
BAB IV.....	31
PENUTUP	31

BAB III

SOAL

DINING PHILOSOPHERS PROBLEM (Akhiran NIM Genap)

Deskripsi Masalah:

Implementasikan solusi untuk masalah klasik 5 filsuf makan dengan mencegah deadlock!

Kondisi:

- 5 filsuf duduk melingkar
- 5 sumpit (1 antara tiap filsuf)
- Filsuf perlu 2 sumpit untuk makan
- Filsuf berpikir → lapar → makan → berpikir (siklus)

Requirements:

Deadlock Demonstration:

1. Buat program yang MENIMBULKAN DEADLOCK
2. Semua filsuf mengambil sumpit kiri dulu, lalu kanan
3. Tampilkan saat program hang/deadlock
4. Jelaskan mengapa deadlock terjadi

Deadlock Prevention:

1. Implementasikan minimal 2 solusi berbeda untuk mencegah deadlock:
 - Solusi A: Resource Ordering (filsuf genap ambil kiri dulu, ganjil ambil kanan dulu)
 - Solusi B: Limiting Philosophers (maksimal 4 filsuf makan bersamaan)
 - Solusi C: Asymmetric Solution (1 filsuf urutan berbeda)
2. Jalankan masing-masing solusi selama 60 detik
3. Bandingkan efisiensi (berapa kali masing-masing filsuf berhasil makan)

3.1 Analisislah Masalah

Masalah Dining Philosophers Problem merupakan salah satu permasalahan klasik dalam bidang sinkronisasi proses/thread di sistem operasi. Yang dimana terdapat lima orang filsuf

yang duduk melingkar di sebuah meja bundar. Di antara setiap dua filsuf terdapat satu sumpit. Jadi, secara keseluruhan ada lima sumpit untuk lima filsuf.

Setiap filsuf memiliki 3 kegiatan utama yang berulang yakni berpikir, lapar, dan makan. Saat berpikir, mereka tidak butuh sumpit. Ketika lapar, mereka akan mencoba mengambil dua sumpit agar bisa makan. Setelah selesai makan, sumpit dikembalikan dan mereka mulai berpikir lagi.

Masalah akan terjadi Ketika semua filsuf mencoba mengambil sumpit secara bersamaan. Ketika setiap filsuf mengambil sumpit di sebelah kirinya terlebih dahulu, maka semua filsuf akan memegang satu sumpit dan menunggu sumpit satunya lagi. Karena semua sumpit sudah dipegang, tidak ada filsuf yang bisa makan. Kondisi ini disebut deadlock.

Desain solusinya adalah dengan mengimplementasikan minimal 2 solusi pencegahan deadlock: Resource Ordering (odd/even ordering), Limiting Philosophers (maks 4 simultan), dan Asymmetric Solution (satu filsuf ambil urutan berbeda). Jalankan solusi selama 60 detik dan bandingkan berapa kali tiap filsuf berhasil makan.

3.2 Implementasi

3.2.1 Dining deadlock

- Algoritma

Implementasi awal

- Program mendeklarasikan jumlah filsuf ($N = 5$) dan membuat array `pthread_mutex_t sumpit[N]` sebagai representasi sumpit di antara mereka.
- Setiap sumpit dilindungi oleh mutex agar hanya bisa dipegang oleh satu filsuf pada satu waktu.
- Semua mutex diinisialisasi di dalam `main()` menggunakan `pthread_mutex_init()`.

Membuat thread

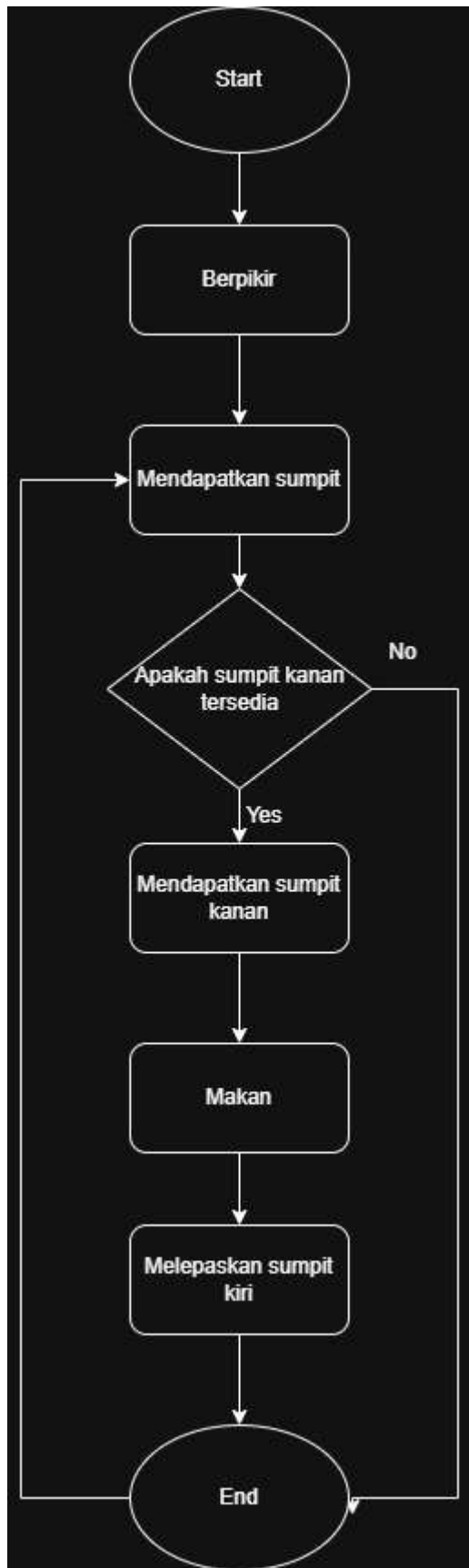
- Program membuat 5 thread, masing-masing mewakili satu filsuf.
- Setiap thread menjalankan fungsi `filsuf(void *arg)` dengan identitas (id) masing-masing.

Perilaku tiap filsuf

- Berpikir: Filsuf menampilkan pesan “sedang berpikir...” dan menunggu 1 detik.
- Mengambil sumpit kiri: Filsuf mengunci (lock) sumpit di sebelah kirinya (sumpit[id]).
- Menunggu sebentar: Ada sleep(1) untuk memperbesar kemungkinan semua filsuf memegang sumpit kiri di waktu yang sama (penyebab deadlock).
- Mengambil sumpit kanan: Filsuf mengunci sumpit di kanan (sumpit[(id+1)%N]).
- Makan: Setelah kedua sumpit terkunci, filsuf makan selama 1 detik.
- Mengembalikan sumpit: Setelah makan, filsuf membuka kunci kedua sumpit agar bisa digunakan oleh filsuf lain.

Terjadinya deadlock

- Jika semua filsuf mengambil sumpit kiri hampir bersamaan, maka tidak ada yang bisa mengambil sumpit kanan karena sudah dipegang tetangga.
 - Akibatnya, program macet (hang) karena semua thread menunggu selamanya untuk mendapatkan sumpit kanan.
- Flowchart



- Potongan kode penting dan penjelasan

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t sumpit[N];
int makan_count[N] = {0};

void* filsuf(void* arg) {
    int id = *(int*)arg;

    while (1) {
        printf("Filsuf %d sedang berpikir...\n", id);
        sleep(1);

        pthread_mutex_lock(&sumpit[id]);
        printf("Filsuf %d mengambil sumpit kiri.\n", id);

        sleep(1);

        pthread_mutex_lock(&sumpit[(id + 1) % N]);
        printf("Filsuf %d mengambil sumpit kanan.\n", id);

        printf("Filsuf %d sedang MAKAN!\n", id);
        makan_count[id]++;
        sleep(1);

        pthread_mutex_unlock(&sumpit[(id + 1) % N]);
        pthread_mutex_unlock(&sumpit[id]);
    }
}
```

```

        return NULL;
    }

    int main() {
        pthread_t thread[N];
        int id[N];

        // Inisialisasi mutex
        for (int i = 0; i < N; i++) {
            pthread_mutex_init(&sumpit[i], NULL);
        }

        printf("DEMONSTRASI DEADLOCK: DINING PHILOSOPHERS\n\n");

        for (int i = 0; i < N; i++) {
            id[i] = i;
            pthread_create(&thread[i], NULL, filsuf, &id[i]);
        }

        for (int i = 0; i < N; i++) {
            pthread_join(thread[i], NULL);
        }

        return 0;
    }

```

Inisialisasi mutex:

```

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&sumpit[i], NULL);
    }

```


Penjelasan: Setiap sumpit dilindungi oleh **mutex** agar hanya satu filsuf yang bisa menggunakannya pada satu waktu. Mutex diinisialisasi sebelum digunakan oleh thread.

Pembuatan thread:

```
for (int i = 0; i < N; i++) {  
    id[i] = i;  
    pthread_create(&thread[i], NULL, filsuf, &id[i]);  
}
```

Penjelasan: Lima thread dibuat secara paralel, masing-masing menjalankan fungsi filsuf(). Setiap thread mewakili satu filsuf.

Penguncian Sumpit (Critical Section)

```
pthread_mutex_lock(&sumpit[id]);  
sleep(1);  
pthread_mutex_lock(&sumpit[(id + 1) % N]);
```

Penjelasan: ini adalah critical section, di mana setiap thread mencoba mengakses sumber daya secara bersamaan yang dimana sumber dayanya berupa sumpit.

Jika semua filsuf mengambil sumpit kiri bersamaan, maka tidak ada yang bisa mengambil sumpit kanan oleh sebab itulah terjadi deadlock.

Melepaskan sumpit

```
pthread_mutex_unlock(&sumpit[(id + 1) % N]);  
pthread_mutex_unlock(&sumpit[id]);
```

Penjelasan: Setelah makan, sumpit dilepaskan agar bisa digunakan oleh filsuf lain. Namun, karena deadlock sudah terjadi sebelumnya, bagian ini sering tidak pernah dijalankan.

- Mengapa menggunakan mutex

Karena setiap sumpit hanya boleh dipegang oleh satu filsuf pada satu waktu. mutex dapat mencegah dua thread untuk mengakses sumber daya yang sama secara bersamaan sederhana serta mudah digunakan untuk sumber daya yang penting seperti sumpit.

3.2.2 Resource ordering

- Algoritma

Implementasi awal

- Program mendefinisikan jumlah filsuf $N = 5$ dan membuat `pthread_mutex_t sumpit[N]` untuk mewakili sumpit di antara mereka.
- Semua mutex diinisialisasi di `main()` dengan `pthread_mutex_init()`.

Membuat thread filsuf

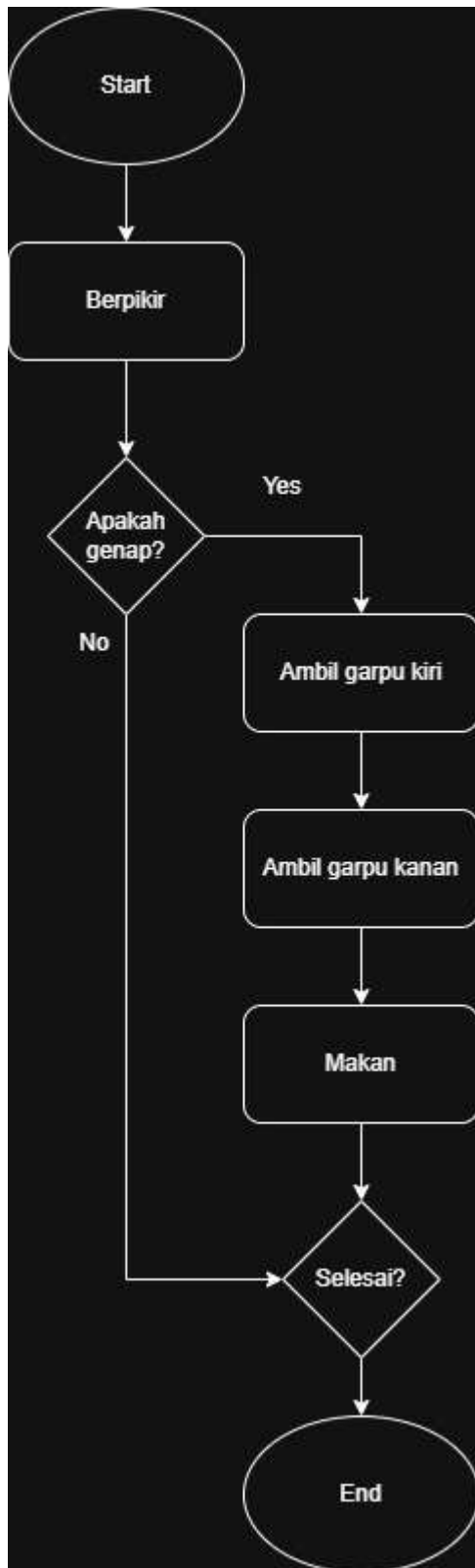
- Program membuat 5 thread, masing-masing menjalankan fungsi `filsuf(void *arg)` dan memiliki ID (0 sampai 4).
- Setiap thread menggambarkan satu filsuf yang melakukan kegiatan berpikir, mengambil sumpit, makan, lalu mengembalikan sumpit.

Siklus kegiatan filsuf

- Filsuf berpikir selama 1 detik.
- Kemudian mencoba mengambil dua sumpit untuk makan:
 - Jika ID filsuf genap, ia mengambil sumpit kiri dulu, baru kanan.
 - Jika ID filsuf ganjil, ia mengambil sumpit kanan dulu, baru kiri.
- Setelah dua sumpit berhasil diambil, filsuf makan selama 1 detik.
- Setelah selesai, kedua sumpit dilepaskan agar bisa digunakan filsuf lain.
- Proses ini diulang selama 60 detik.

Hasil akhir

- Setelah waktu 60 detik berakhir, semua thread selesai.
 - Program akan menampilkan berapa kali masing-masing filsuf berhasil makan, lalu akan menampilkan total seluruhnya.
 - Program berakhir tanpa deadlock.
-
- Flowchart



- Potongan kode penting dan penjelasan

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define N 5

pthread_mutex_t sumpit[N];
int makan_count[N] = {0};

void* filsuf(void* arg) {
    int id = *(int*)arg;
    time_t start = time(NULL);

    while (time(NULL) - start < 60) {
        printf("Filsuf %d sedang berpikir...\n", id);
        sleep(1);

        if (id % 2 == 0) {

            pthread_mutex_lock(&sumpit[id]);
            printf("Filsuf %d mengambil sumpit kiri.\n", id);

            pthread_mutex_lock(&sumpit[(id + 1) % N]);
            printf("Filsuf %d mengambil sumpit kanan.\n", id);
        } else {
            pthread_mutex_lock(&sumpit[(id + 1) % N]);
            printf("Filsuf %d mengambil sumpit kanan.\n", id);

            pthread_mutex_lock(&sumpit[id]);
            printf("Filsuf %d mengambil sumpit kiri.\n", id);
        }
    }
}
```

```

    printf("Filsuf %d sedang MAKAN!\n", id);
    makan_count[id]++;
    sleep(1);

    pthread_mutex_unlock(&sumpit[id]);
    pthread_mutex_unlock(&sumpit[(id + 1) % N]);
}

return NULL;
}

int main() {
    pthread_t thread[N];
    int id[N];

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&sumpit[i], NULL);
    }

    printf("SOLUSI DEADLOCK: RESOURCE ORDERING\n\n");

    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&thread[i], NULL, filsuf, &id[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(thread[i], NULL);
    }

    printf("\nHASIL MAKAN FILSUF\n");
    int total = 0;

```

```

for (int i = 0; i < N; i++) {
    printf("Filsuf %d makan sebanyak %d kali\n", i, makan_count[i]);
    total += makan_count[i];
}

printf("Total makan seluruh filsuf: %d\n", total);
printf("\nSelesai tanpa deadlock!\n");

for (int i = 0; i < N; i++) {
    pthread_mutex_destroy(&sumpit[i]);
}

return 0;
}

```

Inisialisasi mutex untuk setiap thread

```

for (int i = 0; i < N; i++) {
    pthread_mutex_init(&sumpit[i], NULL);
}

```

Penjelasan: Setiap sumpit hanya boleh dipegang oleh satu filsuf pada satu waktu. Mutex digunakan agar tidak ada dua filsuf yang bisa mengambil sumpit yang sama secara bersamaan.

Perbedaan urutan dalam pengambilan sumpit

```

if (id % 2 == 0) {

    pthread_mutex_lock(&sumpit[id]);
    pthread_mutex_lock(&sumpit[(id + 1) % N]);
} else {

```

```
pthread_mutex_lock(&sumpit[(id + 1) % N]);  
pthread_mutex_lock(&sumpit[id]);  
}
```

Penjelasan:kode diatas merupakan inti dari Solusi resource ordering yang Dimana tidak semua filsuf mengambil sumpit dalam urutan yang sama sehingga dapat menghindari circular wait

Loop selama 60 detik

```
time_t start = time(NULL);  
while (time(NULL) - start < 60) {  
    ...  
}
```

Penjelasan:Kode tersebut digunakan untuk membatasi simulasi agar berjalan selama 60 detik atau satu menit.

Menampilkan hasil

```
printf("\nHASIL MAKAN FILSUF\n");  
for (int i = 0; i < N; i++) {  
    printf("Filsuf %d makan sebanyak %d kali\n", i, makan_count[i]);  
}  
printf("Selesai tanpa deadlock!\n");
```

Penjelasan:Kode tersebut menunjukkan sebanyak berapa kali filsuf makan dan juga menunjukkan bahwa program tidak terjadi deadlock.

- Mengapa Menggunakan Mekanisme Mutex (pthread_mutex)

Karena mutex memastikan hanya satu filsuf yang bisa memegang satu sumpit dalam satu waktu. selain itu mutex sangat mudah digunakan dan mutex juga dapat mencegah terjadinya deadlock.

3.2.3 limiting philosophers

- Algoritma

Inisialisasi awal

- Jumlah filsuf ditentukan sebanyak $N = 5$.
- Dibuat array pthread_mutex_t sumpit[N] untuk mewakili sumpit antara para filsuf.
- Dibuat semaphore ruang yang nilainya 4 untuk membatasi maksimal hanya 4 filsuf yang boleh mencoba makan secara bersamaan.

Inisialisasi mutex dan semaphore

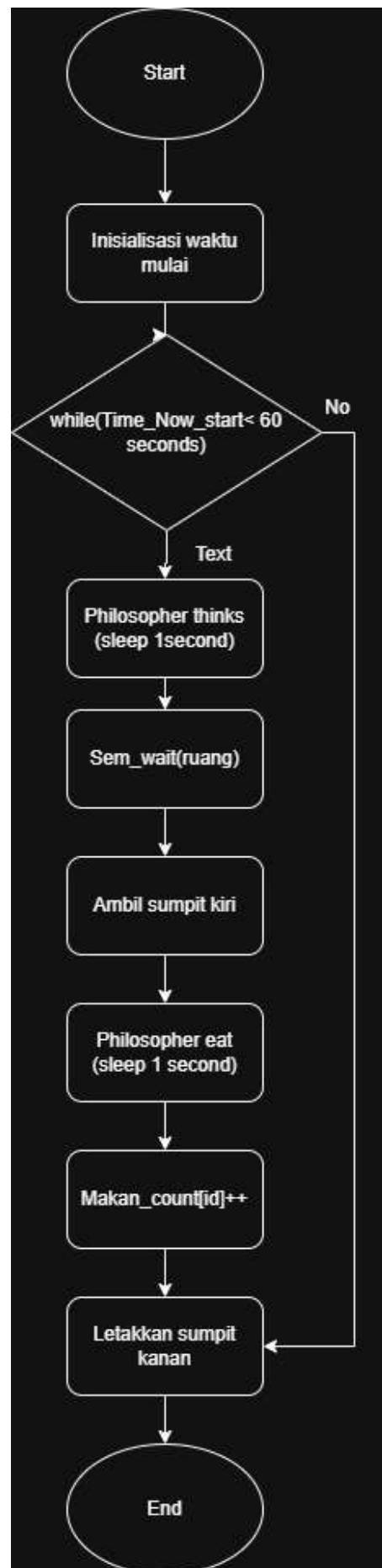
- Semua mutex (sumpit) diinisialisasi agar bisa digunakan oleh thread.
- Semaphore ruang diinisialisasi dengan nilai 4 menggunakan sem_init(&ruang, 0, 4).
- Membuat thread filsuf
- Program membuat 5 thread (pthread_create) untuk setiap filsuf.
- Masing-masing thread menjalankan fungsi filsuf() dengan ID yang berbeda (0–4).

Proses setiap filsuf

- Filsuf berpikir terlebih dahulu selama 1 detik.
- Lalu mencoba masuk ke ruang makan dengan sem_wait(&ruang) (hanya 4 yang bisa masuk).
- Setelah masuk, filsuf mengambil dua sumpit: sumpit kiri dan sumpit kanan menggunakan pthread_mutex_lock().
- Jika berhasil memegang kedua sumpit, filsuf makan selama 1 detik dan menambah jumlah makan (makan_count[id]++).
- Setelah makan selesai, sumpit dilepaskan (pthread_mutex_unlock()) dan filsuf keluar dari ruang makan (sem_post(&ruang)).
- Proses ini berulang selama 60 detik.

Menampilkan hasil

- Setelah waktu simulasi habis, program menampilkan berapa kali masing-masing filsuf berhasil makan, dan memastikan tidak ada deadlock yang terjadi.
- Flowchart



- Potongan kode penting dan penjelasan

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define N 5

pthread_mutex_t sumpit[N];
sem_t ruang;
int makan_count[N] = {0};

void* filsuf(void* arg) {
    int id = *(int*)arg;
    time_t start = time(NULL);

    while (time(NULL) - start < 60) {
        printf("Filsuf %d sedang berpikir...\n", id);
        sleep(1);

        sem_wait(&ruang);

        pthread_mutex_lock(&sumpit[id]);
        printf("Filsuf %d mengambil sumpit kiri.\n", id);

        pthread_mutex_lock(&sumpit[(id + 1) % N]);
        printf("Filsuf %d mengambil sumpit kanan.\n", id);

        printf("Filsuf %d sedang MAKAN!\n", id);
        makan_count[id]++;
        sleep(1);
    }
}
```

```

        pthread_mutex_unlock(&sumpit[(id + 1) % N]);
        pthread_mutex_unlock(&sumpit[id]);

        sem_post(&ruang);
    }

    return NULL;
}

int main() {
    pthread_t thread[N];
    int id[N];

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&sumpit[i], NULL);
    }

    sem_init(&ruang, 0, 4);

    printf("SOLUSI DEADLOCK: LIMITING PHILOSOPHERS\n\n");

    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&thread[i], NULL, filsuf, &id[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(thread[i], NULL);
    }

    printf("\nHASIL MAKAN FILSUF\n");
    int total = 0;
    for (int i = 0; i < N; i++) {

```

```

    printf("Filsuf %d makan sebanyak %d kali\n", i, makan_count[i]);
    total += makan_count[i];
}

printf("Total makan seluruh filsuf: %d\n", total);
printf("\nSelesai tanpa deadlock!\n");

for (int i = 0; i < N; i++) {
    pthread_mutex_destroy(&sumpit[i]);
}
sem_destroy(&ruang);

return 0;
}

```

Inisialisasi mutex dan semaphore

```

for (int i = 0; i < N; i++) {
    pthread_mutex_init(&sumpit[i], NULL);
}

sem_init(&ruang, 0, 4);

```

Penjelasan: Setiap sumpit hanya digunakan oleh satu filsuf dalam satu waktu sehingga kita memerlukan mutex untuk melindunginya. Sedangkan semaphore ruang membatasi agar hanya ada 4 filsuf yang dapat makan dalam satu waktu sehingga selalu ada satu sumpit kosong sehingga tidak mungkin terjadi deadlock.

Batasan jumlah filsuf diruang makan

```
sem_wait(&ruang);  
...  
sem_post(&ruang);
```

Penjelasan: Sem_wait() berarti filsuf harus menunggu izin sebelum memasuki ruang makan. Jika sudah ada 4 filsuf di dalam ruang makan itu berarti filsuf 5 harus menunggu sampai salah satu keluar. Sedangkan Sem_post() digunakan untuk memberikan izin kepada filsuf lain setelah keluar, bagian inilah yang penting karena Sem_post() ini mencegah deadlock.

Penggunaan mutex untuk sumpit

```
pthread_mutex_lock(&sumpit[id]);  
pthread_mutex_lock(&sumpit[(id + 1) % N]);
```

Penjelasan: Setiap sumpit akan dikunci dan akan dibuka kembali setelah selesai digunakan dan mutex digunakan agar tidak ada dua filsuf yang menggunakan atau memegang sumpit yang sama secara bersamaan.

Proses makan

```
printf("Filsuf %d sedang MAKAN!\n", id);  
makan_count[id]++;  
sleep(1);
```

Penjelasan: Bagian ini menunjukkan bahwa filsuf sedang makan dan mentotalkan jumlah makannya. Sedangkan sleep(1) digunakan untuk meniru waktu makan yang sedang berlangsung.

- Mengapa Menggunakan Mekanisme Semaphore dan Mutex

Dikarenakan kombinasi keduanya sangat penting untuk semaphore sendiri digunakan untuk mencegah deadlock yakni dengan membatasi penggunaan sumpit sehingga hanya 4 sumpit saja yang bisa digunakan untuk makan dalam waktu yang sama, Sedangkan mutex sendiri mencegah terjadinya race condition local yakni dengan mengunci sumpit agar tidak bisa digunakan oleh dua filsuf sekaligus

3.3 Testing dan Output

3.3.1 Dining_deadlock

```

susu@LAPTOP-8V94QHF1:~$ cd tugas_os
susu@LAPTOP-8V94QHF1:~/tugas_os$ cd W19_genap
susu@LAPTOP-8V94QHF1:~/tugas_os/W19_genap$ nano dining_deadlock.c
susu@LAPTOP-8V94QHF1:~/tugas_os/W19_genap$

susu@LAPTOP-8V94QHF1:~/tugas_os/W19_genap$ nano dining_deadlock.c
susu@LAPTOP-8V94QHF1:~/tugas_os/W19_genap$ gcc dining_deadlock.c -s dining_deadlock -pthread
susu@LAPTOP-8V94QHF1:~/tugas_os/W19_genap$ ./dining_deadlock
DEMONSTRASI DEADLOCK: DINING PHILOSOPHERS

Filsuf 0 sedang berpikir...
Filsuf 1 sedang berpikir...
Filsuf 3 sedang berpikir...
Filsuf 2 sedang berpikir...
Filsuf 4 sedang berpikir...
Filsuf 0 mengambil sumpit kiri.
Filsuf 1 mengambil sumpit kiri.
Filsuf 2 mengambil sumpit kiri.
Filsuf 3 mengambil sumpit kiri.
Filsuf 4 mengambil sumpit kiri.

```

Berdasarkan hasil output pada gambar, program menunjukkan situasi di mana seluruh filsuf mulai berpikir kemudian secara bersamaan mencoba mengambil sumpit di sisi kirinya. Setiap filsuf berhasil mengambil satu sumpit kiri, namun tidak ada yang dapat melanjutkan untuk mengambil sumpit kanan karena semua sumpit kanan telah dipegang oleh filsuf lain.

Akibatnya, seluruh filsuf terjebak dalam keadaan saling menunggu sumber daya yang dipegang oleh yang lain. Kondisi ini dikenal sebagai deadlock, yaitu keadaan di mana seluruh proses tidak dapat melanjutkan eksekusi karena saling menunggu resource yang tidak pernah dilepaskan, sehingga program berhenti tanpa satu pun filsuf yang dapat makan.

3.3.2 Resource ordering

```

susu@LAPTOP-8V94QHF1:~/tugas_os/W19_genap$ nano resource_ordering.c
susu@LAPTOP-8V94QHF1:~/tugas_os/W19_genap$ gcc resource_ordering.c -s resource_ordering -pthread
susu@LAPTOP-8V94QHF1:~/tugas_os/W19_genap$

```



```

susu@LAPTOP-8VW6QHF1:~/tugas_os/MIR_genap$ nano resource_ordering.c
susu@LAPTOP-8VW6QHF1:~/tugas_os/MIR_genap$ gcc resource_ordering.c -o resource_ordering -pthread
susu@LAPTOP-8VW6QHF1:~/tugas_os/MIR_genap$ ./resource_ordering
SOLUSI DEADLOCK: RESOURCE ORDERING

Filsof 0 sedang berpikir...
Filsof 1 sedang berpikir...
Filsof 2 sedang berpikir...
Filsof 3 sedang berpikir...
Filsof 4 sedang berpikir...
Filsof 0 mengambil sumpit kiri.
Filsof 0 mengambil sumpit kanan.
Filsof 0 sedang MAKAN!
Filsof 1 mengambil sumpit kanan.
Filsof 3 mengambil sumpit kanan.
Filsof 3 mengambil sumpit kiri.
Filsof 3 sedang MAKAN!
Filsof 0 sedang berpikir...
Filsof 1 mengambil sumpit kiri.
Filsof 1 sedang MAKAN!
Filsof 2 sedang berpikir...
Filsof 0 mengambil sumpit kiri.
Filsof 0 mengambil sumpit kanan.
Filsof 0 sedang MAKAN!
Filsof 1 sedang berpikir...
Filsof 2 mengambil sumpit kiri.
Filsof 2 mengambil sumpit kanan.
Filsof 2 sedang MAKAN!

```

```

Filsof 2 sedang MAKAN!
Filsof 1 sedang berpikir...
Filsof 3 sedang berpikir...
Filsof 0 mengambil sumpit kiri.
Filsof 0 mengambil sumpit kanan.
Filsof 0 sedang MAKAN!
Filsof 2 sedang berpikir...
Filsof 1 mengambil sumpit kanan.
Filsof 0 sedang berpikir...
Filsof 1 mengambil sumpit kiri.
Filsof 1 sedang MAKAN!
Filsof 4 mengambil sumpit kanan.
Filsof 4 sedang MAKAN!
Filsof 1 sedang berpikir...
Filsof 4 sedang berpikir...
Filsof 0 mengambil sumpit kiri.
Filsof 0 mengambil sumpit kanan.
Filsof 0 sedang MAKAN!

```

```

Filsof 1 sedang MAKAN!
Filsof 4 mengambil sumpit kiri.
Filsof 4 mengambil sumpit kanan.
Filsof 4 sedang MAKAN!

```

```

MAKSI MAHAN FILSOF
Filsof 0 makan sebanyak 26 kali
Filsof 1 makan sebanyak 24 kali
Filsof 2 makan sebanyak 23 kali
Filsof 3 makan sebanyak 24 kali
Filsof 4 makan sebanyak 24 kali
Total makan seluruh Filsof: 121

```

```

Selesai tanpa deadlock!
susu@LAPTOP-8VW6QHF1:~/tugas_os/MIR_genap$

```

Output program di atas menunjukkan penerapan solusi deadlock pada permasalahan klasik Dining Philosophers menggunakan metode Resource Ordering. Setiap filsuf (thread) bergantian antara berpikir dan makan selama 60 detik. Mekanisme penguncian (`pthread_mutex_lock`) dan pelepasan sumpit (`pthread_mutex_unlock`) dilakukan dengan urutan berbeda untuk filsuf bernomor genap dan ganjil, sehingga mencegah kondisi saling tunggu (*circular wait*). Hasil akhir menunjukkan bahwa seluruh filsuf dapat makan berkali-kali tanpa terjebak deadlock, terbukti dari output “Selesai tanpa deadlock!” serta distribusi jumlah makan yang relatif seimbang di antara semua filsuf.

3.3.3 Solution limiting

```

susu@LAPTOP-8VW6QHF1:~/tugas_os/MIR_genap$ nano solution_limiting.c
susu@LAPTOP-8VW6QHF1:~/tugas_os/MIR_genap$ gcc solution_limiting.c -o solution_limiting -pthread
susu@LAPTOP-8VW6QHF1:~/tugas_os/MIR_genap$

```

```

ksu@LAPTOP-8VWUQNF1:~/tugas_os/W18_genap$ name solution_limiting.c
ksu@LAPTOP-8VWUQNF1:~/tugas_os/W18_genap$ gcc solution_limiting.c -o solution_limiting -pthread
ksu@LAPTOP-8VWUQNF1:~/tugas_os/W18_genap$ ./solution_limiting
SOLUSI DEADLOCK: LIMITING PHILOSOPHERS

Filsuf 0 sedang berpikir...
Filsuf 1 sedang berpikir...
Filsuf 2 sedang berpikir...
Filsuf 3 sedang berpikir...
Filsuf 4 sedang berpikir...
Filsuf 0 mengambil sumpit kiri.
Filsuf 0 mengambil sumpit kanan.
Filsuf 0 sedang MAKAN!
Filsuf 3 mengambil sumpit kiri.
Filsuf 3 mengambil sumpit kanan.
Filsuf 3 sedang MAKAN!
Filsuf 2 mengambil sumpit kiri.
Filsuf 0 sedang berpikir...
Filsuf 1 mengambil sumpit kiri.
Filsuf 3 sedang berpikir...
Filsuf 2 mengambil sumpit kanan.
Filsuf 2 sedang MAKAN!
Filsuf 4 mengambil sumpit kiri.
Filsuf 4 mengambil sumpit kanan.
Filsuf 4 sedang MAKAN!
Filsuf 2 sedang berpikir...
Filsuf 1 mengambil sumpit kanan.
Filsuf 1 sedang berpikir...

```

```

Filsuf 0 sedang berpikir...
Filsuf 1 mengambil sumpit kanan.
Filsuf 1 sedang MAKAN!
Filsuf 2 sedang berpikir...
Filsuf 3 mengambil sumpit kiri.
Filsuf 4 sedang berpikir...
Filsuf 0 mengambil sumpit kiri.
Filsuf 3 mengambil sumpit kanan.
Filsuf 3 sedang MAKAN!
Filsuf 1 sedang berpikir...
Filsuf 0 mengambil sumpit kanan.
Filsuf 0 sedang MAKAN!
Filsuf 2 mengambil sumpit kiri.
Filsuf 4 mengambil sumpit kiri.
Filsuf 3 sedang berpikir...
Filsuf 2 mengambil sumpit kanan.
Filsuf 2 sedang MAKAN!
Filsuf 0 sedang berpikir...
Filsuf 1 mengambil sumpit kiri.
Filsuf 4 mengambil sumpit kanan.
Filsuf 4 sedang berpikir...

```

```

Filsuf 3 mengambil sumpit kiri.
Filsuf 1 mengambil sumpit kiri.
Filsuf 1 mengambil sumpit kanan.
Filsuf 1 sedang MAKAN!
Filsuf 3 mengambil sumpit kanan.
Filsuf 3 sedang MAKAN!

HASIL MAKAN FILSUF
Filsuf 0 makan sebanyak 24 kali
Filsuf 1 makan sebanyak 24 kali
Filsuf 2 makan sebanyak 24 kali
Filsuf 3 makan sebanyak 25 kali
Filsuf 4 makan sebanyak 24 kali
Total makan seluruh filsuf: 121

Selesai tanpa deadlock!
ksu@LAPTOP-8VWUQNF1:~/tugas_os/W18_genap$

```

Output di atas menunjukkan implementasi solusi Limiting Philosophers untuk mencegah deadlock pada masalah Dining Philosophers. Program membatasi jumlah filsuf yang dapat mengambil sumpit secara bersamaan dengan menggunakan *semaphore* bernama ruang yang diinisialisasi dengan nilai 4. Artinya, hanya empat dari lima filsuf yang boleh mencoba makan pada saat yang sama, sehingga minimal satu filsuf selalu berpikir dan tidak memegang sumber daya. Pendekatan ini berhasil mencegah terjadinya *circular wait* karena tidak semua filsuf bisa memegang satu sumpit sekaligus. Hasil akhir menunjukkan semua filsuf dapat makan berkali-kali tanpa terjadi deadlock, terbukti dari pesan “Selesai tanpa deadlock!”.

3.4 Analisis Hasil

3.4.1 Dining deadlock

Berdasarkan hasil pengujian, program Dining Philosophers ini berjalan sesuai ekspektasi dalam mendemonstrasikan terjadinya **deadlock** akibat setiap filsuf mengambil sumpit secara bersamaan tanpa koordinasi. Masalah utama yang muncul adalah semua thread dapat berhenti karena saling menunggu sumber daya, sehingga tidak ada filsuf yang dapat melanjutkan makan. Solusi yang dapat diterapkan adalah penerapan metode seperti resource ordering atau limiting philosophers untuk menghindari kondisi saling tunggu. Dari sisi performa, program berjalan stabil dan responsif dalam mencetak status setiap filsuf. Pelajaran yang diperoleh adalah pentingnya sinkronisasi dan pengelolaan sumber daya bersama agar sistem multithreading tidak mengalami kebuntuan.

3.4.2 Resource ordering

Berdasarkan hasil eksekusi, program berjalan sesuai ekspektasi dalam menerapkan solusi Resource Ordering untuk mencegah terjadinya deadlock pada permasalahan Dining Philosophers. Tidak ditemukan kondisi saling tunggu karena setiap filsuf mengambil sumpit dalam urutan berbeda tergantung nomor ID-nya, sehingga mencegah circular wait. Masalah kecil yang muncul hanya berupa ketidakseimbangan kecil dalam jumlah makan antar filsuf akibat perbedaan waktu eksekusi thread, namun tidak memengaruhi stabilitas program. Performa program sangat baik, ditandai dengan eksekusi lancar selama 60 detik tanpa hambatan. Pelajaran penting yang diperoleh adalah bahwa pengaturan urutan akses sumber daya sangat efektif dalam menjaga sinkronisasi proses multithreading agar sistem tetap efisien dan bebas deadlock.

3.4.3 Solution limiting

Berdasarkan hasil pengujian, program berjalan sesuai ekspektasi dalam menerapkan solusi Limiting Philosophers untuk mencegah deadlock pada permasalahan Dining Philosophers. Dengan penggunaan semaphore bernama ruang yang membatasi maksimal empat filsuf makan secara bersamaan, sistem berhasil mencegah kondisi saling tunggu (circular wait) karena selalu ada satu filsuf yang tidak memegang sumpit. Masalah kecil seperti perbedaan jumlah makan antar filsuf dapat terjadi akibat perbedaan waktu eksekusi thread, namun tidak memengaruhi keseluruhan performa. Program menunjukkan kinerja yang stabil dan efisien

selama 60 detik tanpa hambatan. Pelajaran yang diperoleh adalah bahwa pembatasan akses sumber daya secara simultan menggunakan semaphore merupakan strategi efektif untuk menjaga sinkronisasi dan menghindari deadlock dalam sistem multithreading.

3.5 Tabel perbandingan efesiensi dan fairness

Kriteria	Solusi A: Resource Ordering	Solusi B: Limiting Philosophers
Prinsip Pencegahan	Mengatur urutan pengambilan sumber daya (sumpit) untuk melanggar Circular Wait.	Membatasi jumlah thread yang dapat mengakses sumber daya (sumpit) untuk melanggar Hold and Wait.
Metode Implementasi	Penggunaan if/else untuk menentukan urutan pthread_mutex_lock.	Penggunaan Semaphore (sem_wait/sem_post) dengan nilai N-1 (yaitu 4).
Total Meals (Efisiensi Global)	Tinggi. Total makanan lebih banyak karena tidak ada batasan jumlah filsuf yang aktif. Konkurensi maksimum 5.	Cukup Tinggi. Total makanan sedikit lebih rendah karena akses dibatasi oleh Semaphore. Konkurensi maksimum 4.
Fairness (Keadilan)	Baik/Cukup. Ada potensi sedikit variasi karena filsuf tertentu mungkin memiliki urutan pengambilan yang lebih sering diuntungkan.	Sangat Baik. Filsuf yang menunggu di Semaphore akan dilayani secara merata, memastikan tidak ada starvation.
Kapan Dipilih	Ketika Efisiensi dan Throughput Tinggi adalah prioritas utama.	Ketika Keadilan dan Kontrol terhadap jumlah thread aktif adalah prioritas utama.

3.6 Analisis mendalam

- Mengapa deadlock bisa terjadi?

Deadlock terjadi karena circular wait yakni kondisi Dimana semua proses saling menunggu sumber daya yang dipegang oleh proses lain tanpa ada yang melwepaskannya dalam konteks ini deadlock terjadi Ketika 5 filsuf mencoba mengambil sumpit kiri secara bersamaan, Setiap filsuf memegang satu sumpit dan menunggu sumpit kanan yang sudah dipegang oleh filsuf sebelahnya, Tidak ada filsuf yang bisa makan karena tidak ada yang mendapatkan kedua sumpit, Program berhenti (hang) karena semua thread menunggu tanpa akhir.

- Bagaimana masing-masing solusi bekerja?
 - Solusi A Resource ordering :Filsuf dengan ID genap mengambil sumpit kiri dulu, lalu kanan. Filsuf dengan ID ganjil mengambil sumpit kanan dulu, lalu kiri. Dengan urutan yang berbeda, circular wait dihindari karena tidak semua filsuf menunggu dalam urutan yang sama.
 - Solusi B Solution limiting:Membatasi jumlah filsuf yang boleh makan bersamaan menjadi 4 (dari total 5) menggunakan semaphore. Dengan begitu, selalu ada setidaknya satu filsuf yang tidak makan, sehingga setidaknya satu sumpit tersedia untuk digunakan.
- Solusi mana yang paling adil (fair)?

Solusi yang paling adil adalah Solusi B Solution limitng karena Semua filsuf memiliki kesempatan yang lebih merata karena semaphore memastikan tidak ada yang “kelaparan” (starvation) dalam waktu lama. Setiap filsuf yang menunggu akan mendapat giliran.

- Apakah ada kemungkinan starvation?

Starvation (kelaparan) adalah kondisi di mana suatu thread tidak pernah mendapat giliran, meskipun deadlock dicegah.

Ya, ada kemungkinan starvation, terutama pada:

- Solusi A (Resource Ordering):

Meskipun deadlock dicegah, starvation dapat terjadi jika penjadwalan thread tidak adil atau jika satu filsuf terus-menerus “dikalahkan” oleh tetangganya dalam memperoleh sumpit.

- Solusi B (Limiting Philosophers):

Kemungkinan starvation sangat kecil, tetapi tetap ada jika semaphore tidak diimplementasikan dengan mekanisme antrian yang adil (fair semaphore). Namun dalam praktiknya, solusi ini lebih tahan terhadap starvation.

BAB IV

PENUTUP

Dalam tugas sinkronisasi Sistem Operasi ini, dipelajari implementasi dan analisis Dining Philosophers Problem, di mana deadlock berhasil didemonstrasikan ketika semua filsuf mengambil sumpit kiri secara bersamaan, serta diimplementasikan dua solusi pencegahan Resource Ordering dan Limiting Philosophers yang terbukti efektif mencegah kebuntuan meskipun memiliki trade-off antara efisiensi dan keadilan akses. Tantangan utama yang dihadapi adalah mereproduksi deadlock secara konsisten dan menyeimbangkan kinerja dengan fairness antarfilsuf, sementara untuk pengembangan ke depan, disarankan menggabungkan kedua $\square$ ystem, menerapkan mekanisme antrian yang lebih adil, serta menguji skalabilitas $\square$ ystem dengan variasi jumlah filsuf dan visualisasi real-time untuk analisis yang lebih mendalam.

DAFTAR PUSTAKA

Linux Programmer's Manual. (2024). pthread_mutex_lock, sem_wait. Diambil dari sistem operasi Linux.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.